# A* Path Planning: Lab Reference Sheet

Algorithms, Data Structures, and System Architecture Overview

## 1. The A* Algorithm (Conceptual Logic)

A* balances two main values to decide which grid tile to check next. The node with the absolute lowest $f$ score is always explored first.

- $g$ **(Reality):** The exact accumulated cost to get from the Start tile to the Current tile.
- $h$ **(Heuristic/Guess):** An estimate of the distance from the Current tile to the Goal (e.g., Manhattan or Euclidean distance).
- $f = g + h$ **(Total Score):** The combined priority score.

### The Core Execution Loop

1. **Pop the best node:** Grab the node with the lowest $f$ from your unvisited queue.
2. **Check for Goal:** If this node is the goal, build the path backwards using the `.parent` pointers.
3. **Mark as Visited:** Add the current node to the closed set so you don't evaluate it again.
4. **Explore Neighbors:** For each valid neighboring tile:
   - Calculate its tentative $g$ (current node's $g$ + 1).
   - If it's totally new, calculate its $h$, set its parent, and add to the queue.
   - If it was seen before but this route is faster (lower $g$), update its parent and scores, and push it to the queue again.

## 2. Python Data Structures Cheat Sheet

To implement A* efficiently, you need specialized data structures. Here is how and why to use them in this lab.

| Structure | Purpose in A* | Implementation Examples |
| --- | --- | --- |
| **Min-Heap** `heapq` | The **Open Set** (Unvisited queue). It automatically sorts itself so the item with the lowest $f$-score stays at the top. | `import heapq`<br><br>`open_set = []`<br>`# Pushing an item (sorting relies on f)`<br>`heapq.heappush(open_set, start_node)`<br><br>`# Popping the lowest f-score item`<br>`current = heapq.heappop(open_set)` |
| **Dictionary** `dict` | The **Scoreboard**. Fast lookup to see if a node is already in the Open Set, and what its current best scores are. | `open_dict = {}`<br><br>`# Map a coordinate tuple to the Node`<br>`open_dict[(x, y)] = current_node`<br><br>`# Check if neighbor is already known`<br>`if (nx, ny) in open_dict:`<br>`  existing = open_dict[(nx, ny)]` |
| **Set** `set` | The **Closed Set**. Extremely fast lookup to track tiles we have already completely evaluated. | `closed_set = set()`<br><br>`closed_set.add(current_node)`<br><br>`if neighbor in closed_set:`<br>`  continue # Skip this neighbor` |

## 3. System Architecture: Files and Relationships

Your pathfinder relies on connecting multiple components correctly. Do not try to recalculate grid boundaries manually—use the provided Environment helper methods!

### The Breadcrumb Trail (`.parent`)

The algorithm searches forward, but builds the final route backward. When the `current` tile discovers a `neighbor`, you link them together: `neighbor.parent = current`.

```
[Start Tile: A]
      ↑
[Tile B: .parent = A]
      ↑
[Tile C: .parent = B] ← Goal Found!
```

To reconstruct the final array, loop backward until there are no more parents, then reverse the list:

```
path = []
while current:
    path.append((current.x, current.y))
    current = current.parent
path.reverse() # Flips list to Start -> Goal
return path
```

### Interacting with the SurgicalLabEnvironment

The environment class acts as the "eyes" of your forklift. Use it to scan the area and calculate heuristic math.

- **Finding Neighbors:** Call `self.env.get_neighbors(x, y)`. It returns a list of valid `(nx, ny)` coordinate tuples that are not walls or out-of-bounds.
- **Distance Calculations (Heuristics):** Use `self.env.manhattan_distance(x1, y1, x2, y2)` or `self.env.euclidean_distance(x1, y1, x2, y2)` to calculate your ***h*** score.

# ■ The 30-Minute A* Stress-Test & Heuristic Challenge

## Learning Objectives:

• **Testing & QA:** Learn how to write black-box adversarial test cases to expose algorithmic bias.

• **Algorithm Adaptation:** Modify search behavior by implementing a new geometric heuristic function.

## ■■ Part 1: The Heuristic Code Injection (10 Minutes)

Open your environment.py file. Your task is to write the mathematical formula for **Chebyshev Distance**. Chebyshev distance models 8-way movement where diagonal steps cost the same as cardinal steps (cost = 1).

Task 1.1: Add this method to your SurgicalLabEnvironment class:

```
def chebyshev_distance(self, x1: int, y1: int, x2: int, y2: int) -> float:
    """Calculates 8-way Chebyshev distance"""
    # TODO: Implement the single-line Chebyshev math here
    return max(abs(x1 - x2), abs(y1 - y2))
```

Task 1.2: Update the heuristic checks inside your planner.py:

```
if heuristic_type == 'manhattan':
    start_node.h = self.env.manhattan_distance(start[0], start[1], goal[0], goal[1])
elif heuristic_type == 'chebyshev':
    start_node.h = self.env.chebyshev_distance(start[0], start[1], goal[0], goal[1])
else:
    start_node.h = self.env.euclidean_distance(start[0], start[1], goal[0], goal[1])
```

## ■ Part 2: Adversarial Map Design (10 Minutes)

Boot up the **HTML Visual Map Editor** in your browser. Design a specific 20x15 obstacle layout (under 30% density) that causes **Manhattan Distance** to perform terribly (expand significantly more nodes), while Euclidean distance and/or your newly coded **Chebyshev Distance** breezes through it with optimal efficiency. Export your file as config.py.

## ■ Part 3: The Benchmark & QA Analysis (10 Minutes)

Run your experiment suite in the terminal (python main.py) using your custom map profile and fill out the report tables below.

| Heuristic | Path Length Found | Total Nodes Expanded | Execution Time (ms) |
|---|---|---|---|
| **Manhattan** | 34 | 34 | 0.0 |

| | | | |
|---|---|---|---|
| **Euclidean** | 34 | 34 | 0.0 |
| **Chebyshev** | N / A | N/A | N/A |

**Q1. The 'Why':** Why did Manhattan over-explore so many extra nodes compared to Euclidean/Chebyshev? What specific pattern of walls exploited Manhattan's grid-locked math?

After trying with multiple maps, the best result I have is a tie between Manhattan and Euclidean. This is probably because the forklift cannot actually move diagonally which would be necessary for euclidean to perform better than Manhattan.

**Q2. Heuristic Invalidation:** Even though Chebyshev expands fewer nodes, your current get_neighbors() function only allows 4-way cardinal movement. Because your forklift cannot actually move diagonally, does Chebyshev remain an **admissible heuristic**? Explain the risk of utilizing it here.

Since the Chebyshev function's movement is partially blocked due to the the lack of diagonal movement, Chebyshev distance being a heurisitic that relies on diagonal movements to optimize movement migth actually overestimate the distance and if the heuristic is greater than the actual distance, it cannot be considered admissible.